

Busca em grafos e caminho mais curto

Motivação

- Muitos problemas em computação podem ser expressos com grafos
- Como explorar os nós de um grafo?
 - Como saber se já encontramos todos os nós?
 - Como evitar andar em círculos eternamente?
 - Ex.: $s \rightarrow v_4 \rightarrow v_7 \rightarrow v_2 \rightarrow v_4 \rightarrow v_7 \rightarrow v_2 \rightarrow v_4 \rightarrow v_7 \rightarrow v_2 \rightarrow \dots$
- Os algoritmos de busca em grafos permitem
 - Realizar alguma ação para cada nó e/ou aresta
 - Encontrar caminho mais curto entre dois nós
 - Aprender sobre a estrutura do grafo
 - Ex.: O grafo possui ciclos? Quantos componentes conexos?

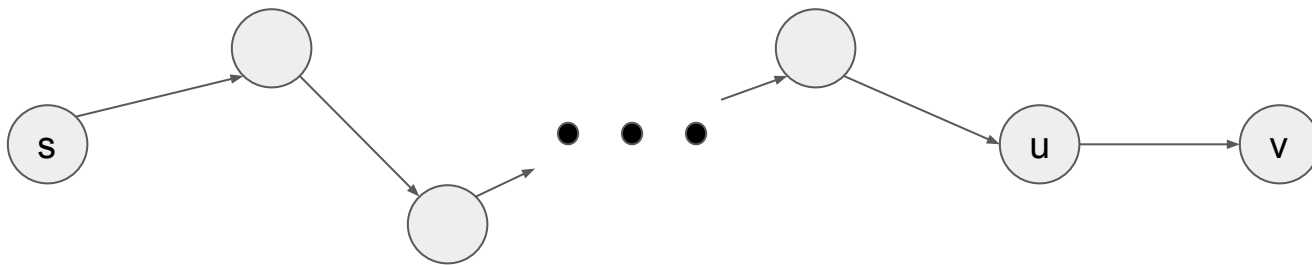
Algoritmo de busca genérico

- Problema de alcançabilidade
 - Pré-condições: entrada é um grafo G (direcionado ou não) e um nó fonte s
 - Pós-condições:
 - Um nó u “alcança” um nó v se existe caminho de u até v em G (notação $u \rightsquigarrow v$)
 - Se G é direcionado, temos que respeitar as direções da arestas
 - Vou assumir G direcionado. Caso contrário basta por arestas nos dois sentidos.
 - A saída é o conjunto de todo nó alcançável a partir de s em G

Algoritmo de busca genérico

- Passos básicos

- Propriedade: Se o nó s alcança o nó u
 - E existe aresta de u para v (denotado com $u \rightarrow v$)
 - Então o nó s alcança o nó v (ou seja, $s \leadsto u$ e $u \rightarrow v$ implica em $s \leadsto v$)



- Se existe aresta de s para os nós em S_1 , então os nós em S_1 são alcançáveis
- Se os nós em S_1 possuem arestas para os nós em S_2 , em os nós em S_2 são alcançáveis
- Assim sucessivamente.

Algoritmo de busca genérico

- Invariante do loop
 - Cada nó tem 2 estados booleanos: “encontrado” e “finalizado”
 - Inicialmente “não encontrado” e “não finalizado”
 - Não volta de “encontrado” p/ “não encontrado”, e de “finalizado” p/ “não finalizado”
 - Basta que nenhum nó seja marcado “não encontrado” ou “não finalizado” fora da inicialização
 - Precisa estar “encontrado” para se tornar “finalizado”
 - IL1: Se um nó é encontrado, então é alcançável a partir de s.
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - Vizinho de um nó u: todo nó v tal que existe aresta de u para v.

Algoritmo de busca genérico

- Código do loop
 - Escolha um nó u qualquer encontrado e não finalizado
 - Percorra cada vizinho v de u
 - Se v não foi encontrado, marque v como encontrado
 - Marque u como finalizado

Algoritmo de busca genérico

algoritmo BuscaGenerica(G,s)

<pre-cond>: G é um grafo (direcionado ou não), e s é um de seus nós

<pos-cond>: saída é conjunto com todo nó alcançável a partir de s

marque todos os nós de G como "não encontrado" e "não finalizado"

marque s como "encontrado"

loop

<inv-loop>: IL1 e IL2.

saia quando nenhum nó está marcado com "encontrado" e "não finalizado"

u = um nó qualquer "encontrado" e "não finalizado"

para cada vizinho v de u em G

se v é "não encontrado"

marque v como "encontrado"

marque u como "finalizado"

retorne os nós "encontrados"

Algoritmo de busca genérico

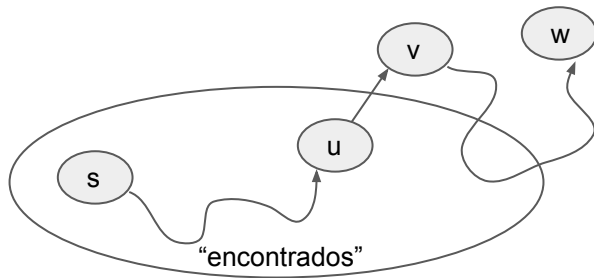
- Mantendo o invariante do loop
 - Assuma que o invariante vale no início do loop
 - IL1': todo nó u encontrado até o início do loop é alcançável a partir de s
 - IL2': todo nó u finalizado até o início do loop tem seus vizinhos encontrados
 - Como a condição de saída não foi satisfeita, existe nó u “encontrado” e “não finalizado”
 - Por IL1', temos que u é alcançável a partir de s (notação: $s \leadsto u$)
 - Como todo nó v indo de “não encontrado” p/ “encontrado” é vizinho de u (notação: $u \rightarrow v$)
 - Concluimos que v também alcançável a partir de s (pois $s \leadsto u$ e $u \rightarrow v$ implica em $s \leadsto v$)
 - Isso garante IL1'', pois todos “encontrados” permanecem alcançáveis
 - Por IL2', os finalizados anteriormente já possuem seus vizinhos como “encontrado”
 - O nó u é o único que se torna “finalizado” na iteração
 - Pelo código do loop, todos seus vizinhos ficam com estado “encontrado”
 - Com isso concluimos IL2''

Algoritmo de busca genérico

- Medida de progresso
 - Se usar o número de nós encontrados, podemos ter iteração sem avanço
 - Basta que todos os vizinhos de u já estejam marcados com “encontrado”
 - Melhor usar o número de finalizados, pois garantimos avançar em cada iteração
 - Com isso sabemos que o número de iterações é no máximo o número de nós de G
- Condição de saída
 - Só é possível avançar enquanto houver “encontrado” que ainda não foi “finalizado”
- Código pré-loop satisfaz o invariante do loop?
 - Marca todos os nós como “não finalizado”, e apenas o nó s como “encontrado”
 - IL1: o nó s é alcançável partir de s
 - IL2: verdadeiro por vacuidade, pois nenhum nó está “finalizado”

Algoritmo de busca genérico

- Pós-condições são satisfeitas? (alcançáveis = encontrados?)
 - O código pós-loop apenas retorna os nós encontrados
 - Como acabamos de sair do loop, podemos assumir que IL1 e IL2 são verdadeiras
 - IL1 garante que os encontrados são alcançáveis. Resta mostrar o contrário.
 - Assuma por contradição que existe nó *w* alcançável a partir de *s* que não foi encontrado
 - Considere um caminho *P* qualquer de *s* até *w*
 - Seja *v* o primeiro nó não encontrado em *P*,
e *u* o predecessor de *v*
 - Ou seja, *u* foi encontrado.
 - Pela condição de saída temos que todo nó encontrado está finalizado
 - Com isso concluímos que *u* está finalizado
 - Então, por IL2 temos que todos vizinho de *u* foi encontrado
 - Contradição, pois assumimos que o vizinho *v* não foi encontrado.



Algoritmo de busca genérico

- Tempo de execução
 - Entrada tem n nós e m arestas
 - Código pré-loop é $\theta(n)$
 - Retorno dos encontrados é $\theta(1)$ (basta armazenar cada nó quando encontrar)
 - Uma iteração para cada nó
 - Cada nó tem seus vizinhos explorados uma única vez
 - Ou seja, cada aresta é explorada exatamente uma vez
 - Vou assumir que o tempo gasto em cada vizinho é constante
 - Verdade p/ busca em largura/profundidade. Falso para Dijkstra.
 - Portanto, o laço e o algoritmo são $\theta(n + m)$ (se G é uma lista de adjacência)
 - Obs.: se G estiver em matriz de adjacência, percorrer os vizinhos de um nó seria $\theta(n)$
 - Resultando em algoritmo $\theta(n^2)$

Algoritmo de busca genérico

- Tempo de execução
 - Grafos esparsos: $m \in \theta(n)$
 - Busca é $\theta(n+m) = \theta(n)$
 - Grafos densos: $m \in \theta(n^2)$
 - Busca é $\theta(n+m) = \theta(n^2)$

Algoritmo de busca genérico

- Ordem de escolha dos nós
 - Fila (busca em largura)
 - Distância de u : menor número de arestas dentre os caminhos de s até u
 - Visita nós com distância 1, depois distância 2, assim sucessivamente
 - Permite então encontrar o caminho mais curto de s para todos os nós
 - Pilha (busca em profundidade)
 - Afasta-se o mais rapidamente possível do nó s , e faz backtrack quando não é possível
 - Fila de prioridade (caminho mais curto ponderado)
 - Distância de u : menor soma dos pesos das arestas dentre os caminhos de s até u
 - Prioridade é uma estimativa da distância de s até u
 - Estimativa é atualizada sempre que uma aresta para u é explorada
 - Nó de menor prioridade é o estimado mais próximo de s , dentre os não finalizados
 - Este é o algoritmo de Dijkstra para caminhos mais curtos em grafos ponderados

Exemplo: busca em largura

Fila:

s

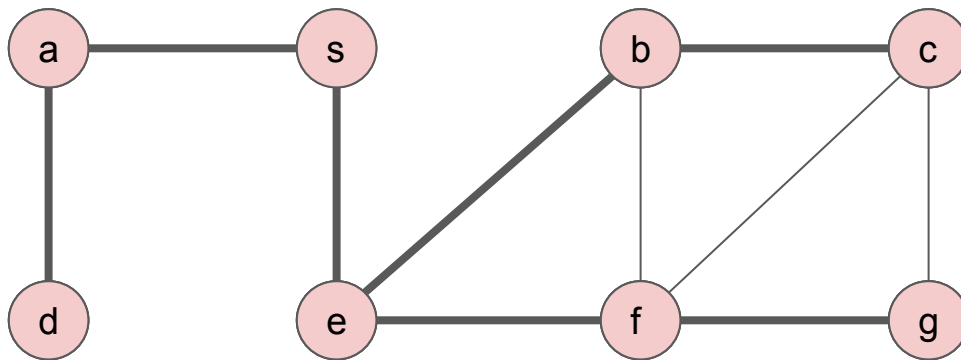
e a

a b f

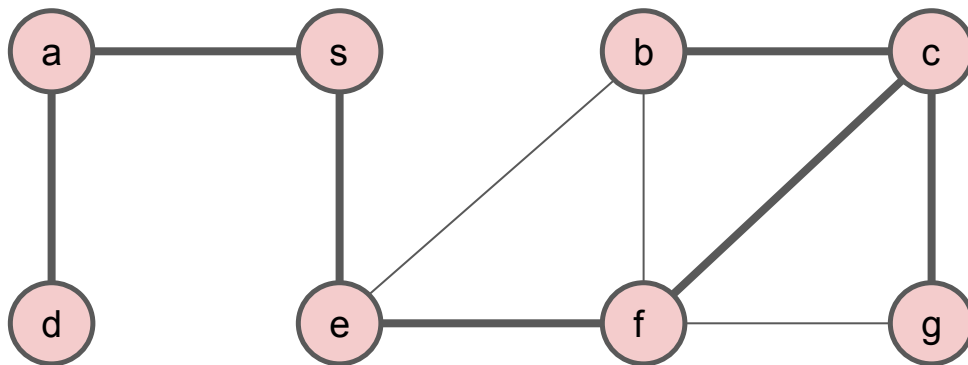
b f d

f d c

d c g



Exemplo: busca em profundidade



Pilha:
s
vazia
s e
s
s e f
s e
s e f c
s e f
s e f c g
s e f c
s e f
s e f c b
s e f c
s e f
s e
s
vazia
s a
s
s a d
s a
s
vazia

Caminho mais curto
com busca em largura

Caminho mais curto com busca em largura

- Problema de caminho mais curto
 - Pré-condições: G é um grafo (direcionado o não) e s é um nó qualquer de G .
 - Pós-condições: saída é um $d(v)$ e um $\pi(v)$ para cada nó v de G tal que
 - $d(v)$ é a distância de s até v em G
 - Distância: menor número de arestas dentre os caminhos de s até v
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mais curtos de s para todos os outros nós
 - Ou seja, seguindo π é possível obter um caminho mais curto até qualquer nó
- Encontrar caminho mais curto entre dois nós é um problema de otimização
 - Instância: grafo G e dois nós s e t de G
 - Solução: caminho P de s até t em G
 - Medida de sucesso: número de arestas em P
 - Objetivo: encontrar um caminho com o menor número de arestas entre s e t
 - Força bruta é inviável: existe uma quantidade exponencial de possíveis caminhos

Caminho mais curto com busca em largura

- Como provar que um caminho mais curto até v tem tamanho $d(v)$?
 - Mostre que existe caminho com tamanho $d(v)$, identificando $d(v)$ arestas
 - Mostre que não existe caminho de s até v com menos que $d(v)$ arestas
 - Inviável testar todos os caminhos possíveis
 - Vamos usar o fato dos nós serem encontrados em ordem não decrescente de distância
 - Então quando o nó é encontrado, sabemos que não há caminho mais curto

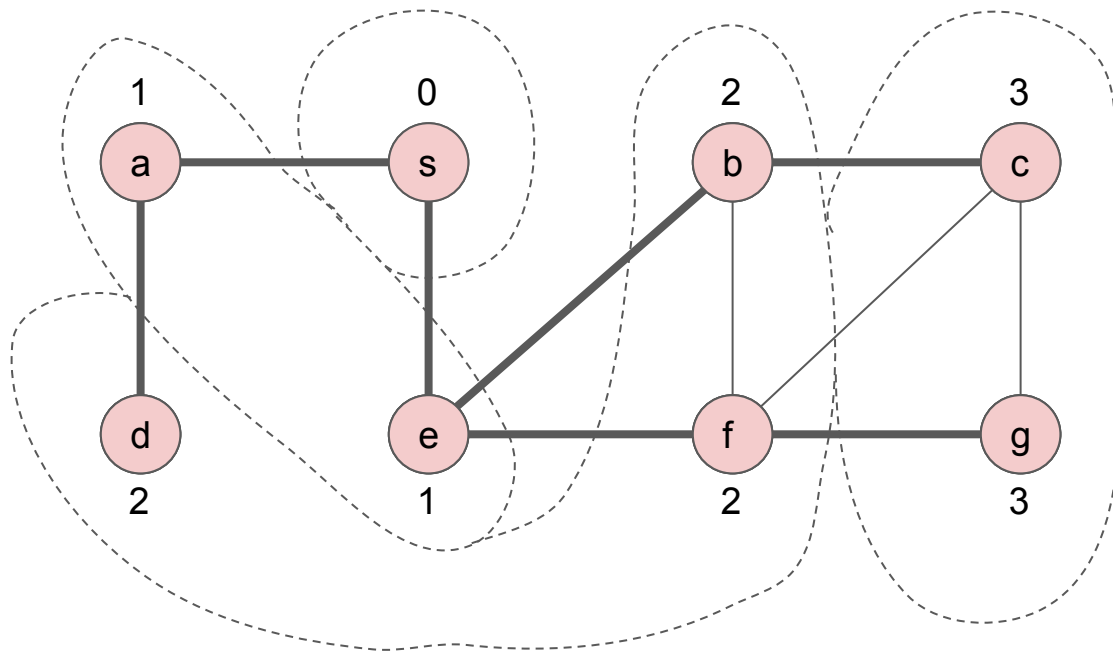
Caminho mais curto com busca em largura

- Invariante do loop
 - Seja V_j o conjunto de nós a uma distância j de s
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
 - Ou seja, todos os nós com distância menor que j foram encontrados

Caminho mais curto com busca em largura

Veremos que IL3 pode ser obtido armazenando os encontrados em uma fila (busca em largura)

Fila:
s
e a
a b f
b f d
f d c
d c g



Caminho mais curto com busca em largura

- Código do loop
 - Remova o nó u do início da fila
 - Percorra cada vizinho v de u
 - Se v não foi encontrado
 - Marque v como encontrado
 - Faça $d(v) = d(u) + 1$ e $\pi(v) = u$
 - Insira v no final da fila
 - O estado “finalizado”/“não finalizado” é gerenciado através da fila
 - Não finalizado: nós encontrados que estão na fila
 - Finalizado: nós encontrados que não estão na fila

Caminho mais curto com busca em largura

algoritmo BuscaLargura(G, s)

<pre-cond>: G é um grafo (direcionado ou não), e s é um de seus nós

<pos-cond>: para cada nó v , distância $d(v)$ e pai $\pi(v)$ na árvore de cam. mín.

 encontrado(v) == True sse v é alcançável a partir de s .

para todo nó u de G

 encontrado(u) = False; $d(u) = \infty$; $\pi(u) = \text{nulo}$

encontrado(s) = True; $d(s) = 0$

Q = fila contendo apenas o nó s // Q contém nós encontrados e não finalizados

loop

 <inv-loop>: IL1, IL2 e IL3.

 saia quando a fila Q estiver vazia // não há encontrado não finalizado

$u = Q.\text{dequeue}()$

 para cada vizinho v de u em G // percorra a lista de adj de u

 se encontrado(v) == False

 encontrado(v) = True; $d(v) = d(u) + 1$; $\pi(v) = u$

$Q.\text{enqueue}(v)$

Caminho mais curto com busca em largura

- Mantendo o invariante do loop
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
- Assuma IL1', IL2' e IL3' no início do laço, e que existe nó u no início da fila
- Mostrar P: “se $u \in V_k$ então todos os nós de V_k já foram encontrados”
 - Quando u foi encontrado, por IL3' todos em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados
 - Então todos eles entraram na fila antes de u
 - Como u é o 1o da fila nesta iteração, todos eles já finalizaram
 - Então, por IL2' temos que todos os vizinhos de V_{k-1} já foram encontrados
 - E os nós em V_k são vizinhos dos nós em V_{k-1}

Caminho mais curto com busca em largura

- Mantendo o invariante do loop
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
- Assuma IL1', IL2' e IL3' no início do laço, e que existe nó u no início da fila
- P: “se $u \in V_k$ então todos os nós de V_k já foram encontrados”
- Mostrar IL1”. Quando um vizinho v de u é encontrado na iteração
 - Por IL1' sabemos que $d(u)$ é a distância de s até u
 - Como existe aresta de u para v , e caminho de tamanho $d(u)$ de s até u
 - Então existe caminho de tamanho $d(u) + 1$ de s até v , onde $\pi(v)=u$
 - Resta verificar se existe caminho de s até v com tamanho menor que $d(u) + 1$
 - Por IL3' e P todo caminho de tamanho até $d(u)$ já havia sido encontrado

Caminho mais curto com busca em largura

- Mantendo o invariante do loop
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
 - Assuma IL1', IL2' e IL3' no início do laço, e que existe nó u no início da fila
 - P: “se $u \in V_k$ então todos os nós de V_k já foram encontrados”
 - Mostrar IL2’
 - Por IL2', os finalizados anteriormente tem os vizinhos encontrados
 - Nesta iteração finalizamos apenas u , mas só depois de encontrar todos os vizinhos

Caminho mais curto com busca em largura

- Mantendo o invariante do loop
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
 - Assuma IL1', IL2' e IL3' no início do laço, e que existe nó u no início da fila
 - P: “se $u \in V_k$ então todos os nós de V_k já foram encontrados”
 - Mostrar IL3’
 - Por IL1”, todo nó v encontrado na iteração tem distância $d(u)+1$
 - Por IL3’ e P todo caminho de tamanho até $d(u)$ já havia sido encontrado

Caminho mais curto com busca em largura

- Estabelecendo o invariante do loop (código pré-loop)
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
- IL1: s é o único encontrado, e tem corretamente $d(s) = 0$ e $\pi(s)$ nulo
- IL2: verdadeiro por vacuidade, pois inicialmente não há nó finalizado
 - O único encontrado (s) está na fila
- IL3: verdadeiro, pois não existe nó com distância menor que 0

Caminho mais curto com busca em largura

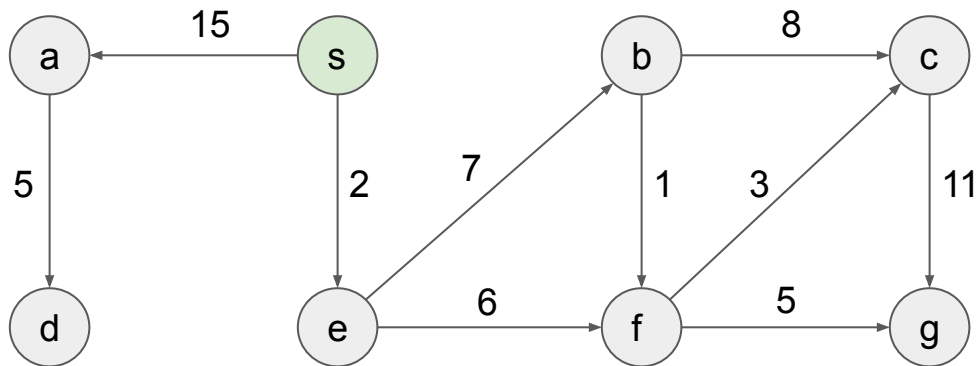
- Estabelecendo as pós-condições
 - IL1: Se v é encontrado, então
 - $d(v)$ é a distância de s até v em G
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mínimos até s
 - IL2: Se um nó é finalizado, então todos seus vizinhos já foram encontrados.
 - IL3: Quando $v \in V_k$ é encontrado, todos os nós em $V_0, V_1, \dots, V_{k-1}$ já haviam sido encontrados.
 - Pela discussão do algoritmo de busca genérica, todo alcançável é encontrado
 - Por IL1, d e π fornecem a informação correta para os nós alcançáveis (encontrados)
 - Para os nós não alcançáveis,
 - os valores $d(u) = \infty$ e $\pi(v) = \text{nulo}$ recebidos na inicialização são mantidos

Caminho mais curto com algoritmo de Dijkstra

Caminho mais curto em grafos ponderados (Dijkstra)

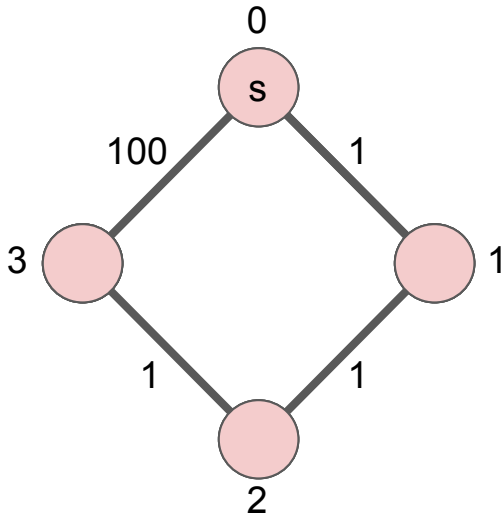
- Especificação

- Pré-condições: G é um grafo (direcionado ou não) e s um nó qualquer de G .
 - Cada aresta (u,v) tem um peso não negativo $w[u,v]$
- Pós-condições: saída é um $d(v)$ e um $\pi(v)$ para cada nó v de G tal que
 - $d(v)$ é a distância de s até v em G
 - Distância: menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é o pai de v em uma árvore de caminhos mais curtos de s para todos os outros nós



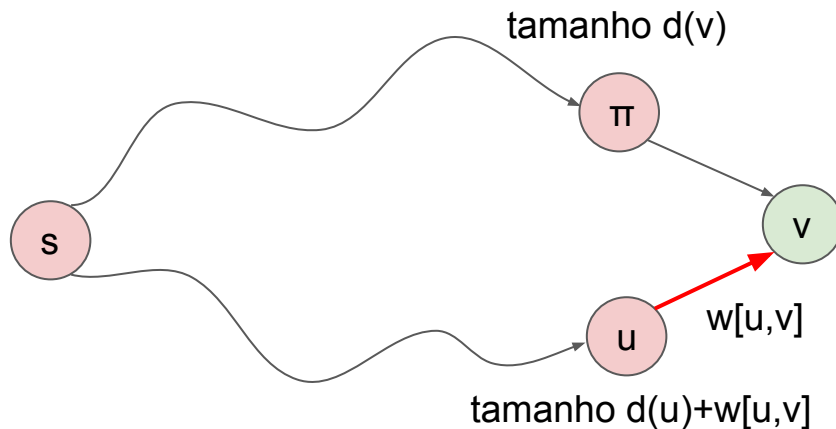
Caminho mais curto em grafos ponderados (Dijkstra)

- Como provar que um caminho mais curto até v tem tamanho $d(v)$?
 - Mostre um caminho cuja soma das arestas vale $d(v)$
 - Mostre que não existe caminho de s até v com soma dos pesos das arestas menor que $d(v)$
 - Nos grafos não ponderados encontramos nós em ordem não decrescente de distância
 - Agora vamos finalizar os nós em ordem não decrescente de distância



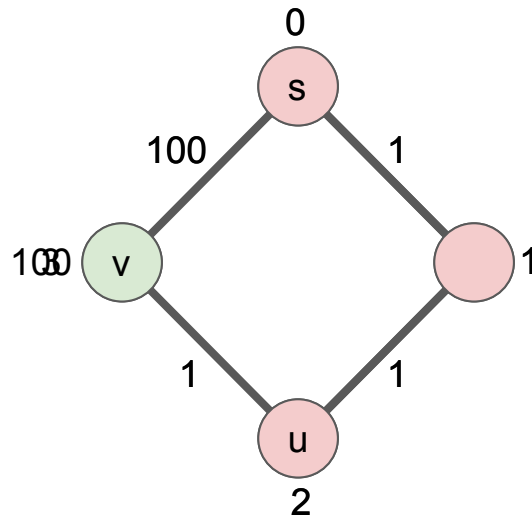
Caminho mais curto em grafos ponderados (Dijkstra)

- Atualização das estimativas $d(v)$ (operação de relaxação)
 - Assuma que no início de cada iteração
 - $d(u)$ é o comprimento do caminho mais curto até u usando apenas nós finalizados
 - $\pi(u)$ é o predecessor em um caminho mais curto até u usando apenas nós finalizados
 - Se existe aresta de u para v , podemos aplicar a operação de relaxação abaixo:
se $d(v) > d(u) + w[u, v]$
 $d(v) = d(u) + w[u, v]$
 $\pi(v) = u$



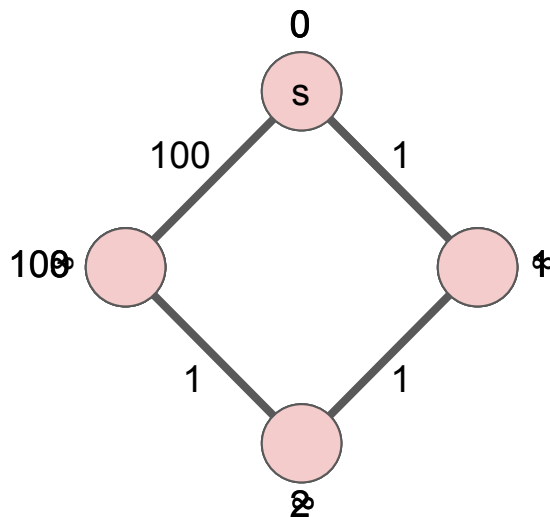
Caminho mais curto em grafos ponderados (Dijkstra)

- Atualização das estimativas $d(v)$ (operação de relaxação)
 - Assuma que no início de cada iteração
 - $d(v)$ é o comprimento do caminho mais curto até v usando apenas nós finalizados
 - $\pi(v)$ é o predecessor em um caminho mais curto até v usando apenas nós finalizados
 - Se existe aresta de u para v , podemos aplicar a operação de relaxação abaixo:
 - se $d(v) > d(u) + w[u, v]$
 - $d(v) = d(u) + w[u, v]$
 - $\pi(v) = u$



Caminho mais curto em grafos ponderados (Dijkstra)

- Como escolher o próximo nó para finalizar?
 - Vamos usar o nó v com menor estimativa $d(v)$ dentre os nós encontrados e não finalizados



Ao invés de armazenar os encontrados e não finalizados em uma fila, vamos armazená-los em uma fila de prioridade.

Prioridade: estimativa $d(v)$
Extraímos o de menor prioridade.

No início todo nó é inserido na FP com prioridade $d(v) = \infty$ (exceto s).
(todos os não finalizados)

Caminho mais curto em grafos ponderados (Dijkstra)

- Invariante do loop
 - “Caminho de finalizados”: quando todas as arestas partem de nós finalizados
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
- Código do loop
 - Para cada nó u de menor prioridade extraído da fila de prioridade
 - Relaxe todas as arestas que saem de u
 - (por IL1, será em efeito se apontar para um nó já finalizado)

Caminho mais curto em grafos ponderados (Dijkstra)

algoritmo Dijkstra(G, s, w)

<pre-cond>: G é um grafo (direcionado ou não), e s é um de seus nós.

$w[u, v]$ é o peso de cada aresta (u, v) .

<pos-cond>: para cada nó v , distância $d(v)$ e pai $\pi(v)$ na árvore de cam. mín.

crie min-heap Q contendo o nó s com prioridade 0,

e todos os outros nós com prioridade ∞

loop

<inv-loop>: IL1 e IL2.

saia quando Q estiver vazia

$u = Q.\text{extract_min}()$

para cada vizinho v de u em G // percorra a lista de adj de u

se $d(v) > d(u) + w[u, v]$

$d(v) = d(u) + w[u, v]$

$\pi(v) = u$

$Q.\text{update_priority}(v, d(v))$

Caminho mais curto em grafos ponderados (Dijkstra)

- Mantendo o invariante do loop
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
 - Notação:
 - $|P|$ detona as somas dos pesos das arestas no caminho P (“tamanho” do caminho P)
 - $d'(v)$: valor de $d(v)$ no início da iteração
 - $d''(v)$: valor de $d(v)$ no final da iteração
 - Se usar apenas $d(v)$, significa que $d(v)$ não foi alterado na iteração
 - IL1' e IL2': invariante do loop no início da iteração
 - IL1'' e IL2'': invariante do loop no final da iteração

Caminho mais curto em grafos ponderados (Dijkstra)

- Mantendo o invariante do loop
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
 - Pela operação de relaxação, temos que π fornece caminho de tamanho $d(v)$ de s para v
 - Basta focar a demonstração nos valores de d

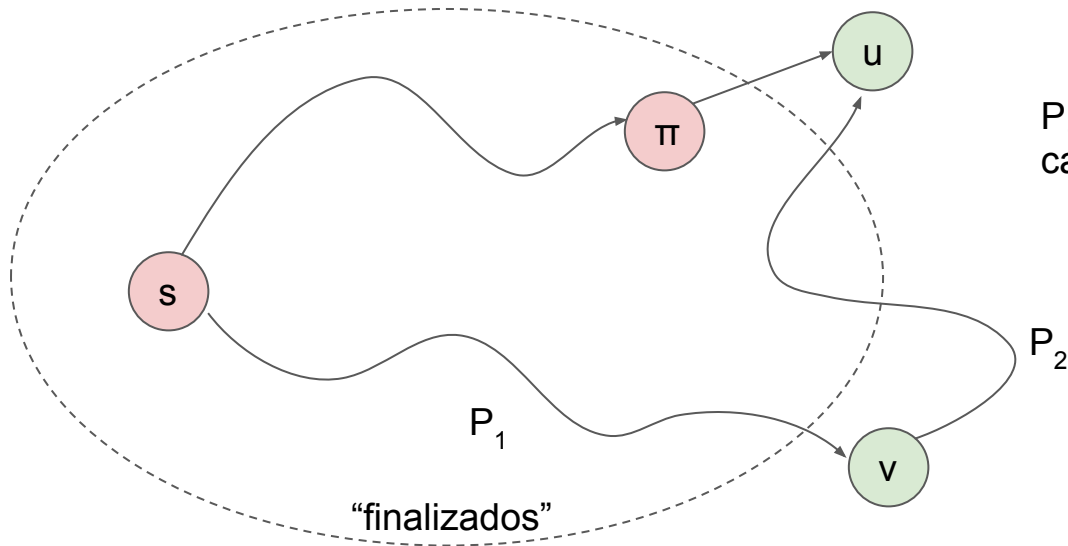
Caminho mais curto em grafos ponderados (Dijkstra)

- Mantendo o invariante do loop
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
 - Assuma IL1' e IL2', e que Q não está vazia
 - O nó u torna-se finalizado ao sair de Q , e é o único nó a sair de Q na iteração
 - IL1'': mostrar que $d(u)$ é o tamanho do caminho mais curto de s até u
 - Note que isso precisa ser verdade no início da iteração, pois não altera $d(u)$
 - Ou seja, o nó escolhido para sair da FP já tem o valor final de d
 - Por contradição, assumo que existe caminho mais curto P menor que $d(u)$

Caminho mais curto em grafos ponderados (Dijkstra)

P tem algum nó v ($\neq u$) não finalizado, pois caso contrário $|P|=d(u)$ por IL2'.

Seja v ($\neq u$) o primeiro nó não finalizado em P.
No caminho P, P_1 vai de s até v , e P_2 vai de v até u .



P_1 é um caminho mais curto de s até v , pois caso contrário P não seria mais curto até u .

Então, por IL2', $d(v)$ é o tamanho de P_1 .

$d(u) \leq d(v)$, pois u e v estão na heap, mas u é escolhido para sair da heap.

Como $|P| = |P_1| + |P_2| = d(v) + |P_2| < d(u)$, concluímos que $|P_2| < d(u) - d(v) \leq 0$ (contradição, pois os pesos são não negativos).

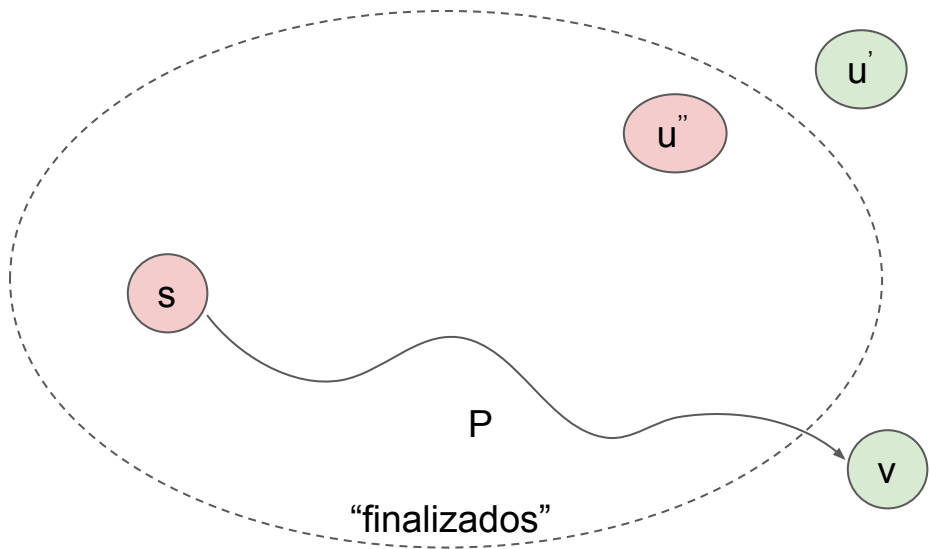
Caminho mais curto em grafos ponderados (Dijkstra)

- Mantendo o invariante do loop
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
 - Resta mostrar IL2'': para todo v não finalizado no final da iteração
 - $d''(v)$ é o tamanho do caminho de finalizados mais curto de s até v
 - Como apenas o nó u foi finalizado na iteração
 - Mostrar que a inclusão de u nos finalizados não reduz o tamanho do caminho de finalizados mais curto de s até um nó v qualquer não finalizado
 - Ou seja, se P é caminho de finalizados de s até v podendo incluir u , $|P| \geq d''(v)$

Caminho mais curto em grafos ponderados (Dijkstra)

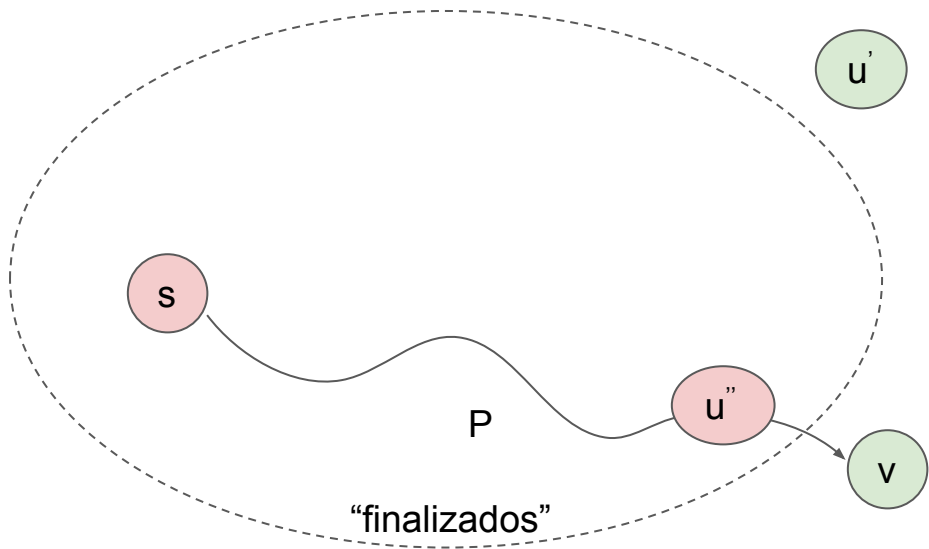
- Mantendo o invariante do loop
 - Caso I: P não contém u
 - Caso II: P contém u
 - Caso IIa: u é predecessor de v em P
 - Caso IIb: u não é predecessor de v em P

Caminho mais curto em grafos ponderados (Dijkstra)



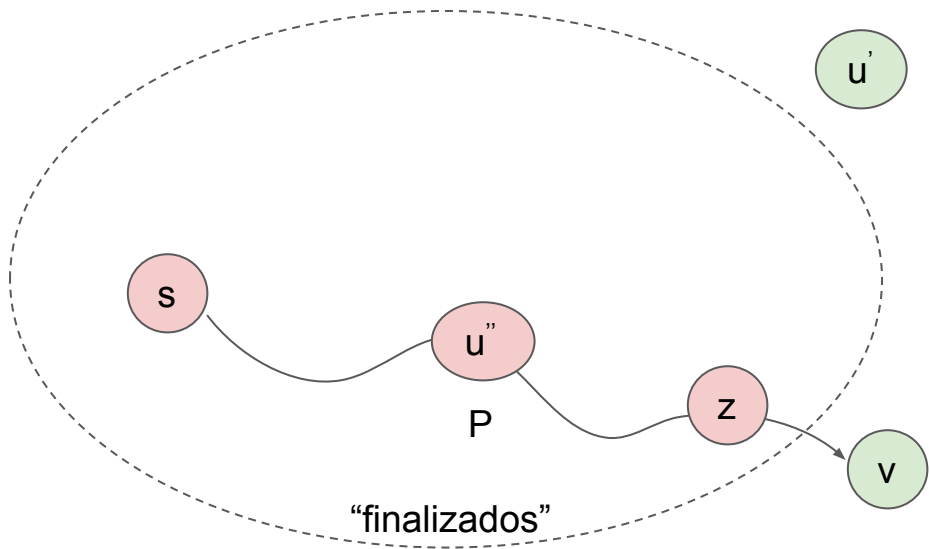
- Caso I: P não contém u
 - Por IL2', $d'(v)$ é o tamanho do caminho de finalizados mais curto de s até v (sem o nó u)
 - Então, $|P| \geq d'(v) \geq d''(v)$.

Caminho mais curto em grafos ponderados (Dijkstra)



- Caso II: P contém u
 - Caso IIa: u é predecessor de v em P
 - Por IL2', $d(u)$ é o tamanho do caminho de finalizados mais curto de s até u .
 - Como (u,v) foi relaxada na iteração
 - $d''(v) \leq d(u) + w[u,v] \leq |P|$.

Caminho mais curto em grafos ponderados (Dijkstra)



- Caso II: P contém u
 - Caso IIb: u não é predecessor de v em P
 - Seja z ($\neq u$) o predecessor de v em P .
 - Quando um nó finaliza, o caminho de finalizados mais curto torna-se o caminho mais curto
 - Como z já estava finalizado no início da iteração
 - Por IL1' existe caminho mais curto até z que não inclui u
 - Pois u não estava finalizado quando z finalizou
 - $|P| \geq d(z) + w[z, v]$
 - Como (z, v) foi relaxada
 - $|P| \geq d(z) + w[z, v] \geq d'(v) \geq d''(v)$

Caminho mais curto em grafos ponderados (Dijkstra)

- Estabelecendo o invariante do loop (código pré-loop)
 - IL1: para cada nó finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os caminhos de s até v
 - $\pi(v)$ é predecessor de v em um caminho mais curto de s até v
 - IL2: para cada nó **não** finalizado v
 - $d(v)$ é a menor soma dos pesos dentre todos os **caminhos de finalizados** de s até v
 - $\pi(v)$ é predecessor de v em um **caminho de finalizados** mais curto de s até v
 - Código pré-loop: crie min-heap Q contendo o nó s com prioridade 0, e todos os outros nós com prioridade ∞
 - IL1 é satisfeito pois nenhum nó está finalizado
 - IL2: como nenhum nó está finalizado
 - Não é possível alcançar nó diferente de s com caminho de finalizados
 - Então temos que ter $d(s) = 0$, $d(v) = \infty$ e $\pi(v) = \text{nulo}$ para $v \neq s$
 - $\pi(s) = \text{nulo}$, pois s é a raiz da árvore de caminhos mais curtos

Caminho mais curto em grafos ponderados (Dijkstra)

- Estabelecendo as pós-condições
 - O código pós-loop apenas retorna d e π
 - Pela condição de saída a FP está vazia (todos os nós foram finalizados)
 - Então, por IL1, os valores de d e π estão calculados corretamente para todo nó

Caminho mais curto em grafos ponderados (Dijkstra)

- Tempo de execução

algoritmo Dijkstra(G, s, w)

crie min-heap Q contendo o nó s com prioridade 0,
e todos os outros nós com prioridade ∞

$\theta(n)$ (operação `make_heap`)

loop

n iterações

saia quando Q estiver vazia

$u = Q.\text{extract_min}()$

$\theta(\log n)$ por iteração

para cada vizinho v de u em G

$\theta(m)$

se $d(v) > d(u) + w[u, v]$

$d(v) = d(u) + w[u, v]$

$\theta(1)$ por aresta

$\pi(v) = u$

$Q.\text{update_priority}(v, d(v))$

$\theta(\log n)$ por aresta

Total: $\theta(n \log n + m \log n) = \theta((n+m) \log n)$